ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/226987370

Using Vanishing Points for Camera Calibration and Coarse 3D Reconstruction from A Single Image

Article *in* The Visual Computer · November 2000

DOI: 10.1007/PL00013394 · Source: DBLP

CITATIONS
174

READS
8,819

4 authors:

Erwan Guillou
Claude Bernard University Lyon 1
29 PUBLICATIONS **529** CITATIONS

SEE PROFILE

Daniel Meneveaux
Université de Poitiers
79 PUBLICATIONS **666** CITATIONS

SEE PROFILE

Eric Maisel
Brest National Engineering School
34 PUBLICATIONS **331** CITATIONS

SEE PROFILE

Kadi Bouatouch
University of Rennes
170 PUBLICATIONS **2,292** CITATIONS

SEE PROFILE

All content following this page was uploaded by Kadi Bouatouch on 28 July 2014.

The user has requested enhancement of the downloaded file.

The Visual Computer

Using vanishing points for camera calibration and coarse 3D reconstruction from a single image

E. Guillou, D. Meneveaux,
E. Maisel, K. Bouatouch

IRISA, Campus de Beaulieu, 35042 Rennes cedex, France
e-mail: {eguillou, dmenevea, maisel, kadi}@irisa.fr

In this paper, we show how to calibrate a camera and to recover the geometry and the photometry (textures) of objects from a single image. The aim of this work is to make it possible walkthrough and augment reality in a 3D model reconstructed from a single image. The calibration step does not need any calibration target and makes only four assumptions: (1) the single image contains at least two vanishing points, (2) the length (in 3D space) of one line segment (for determining the translation vector) in the image is known, (3) the principle point is the center of the image, and (4) the aspect ratio is fixed by the user. Each vanishing point is determined from a set of parallel lines. These vanishing points help determine a 3D world coordinate system R_O. After having computed the focal length, the rotation matrix and the translation vector are evaluated in turn for describing the rigid motion between R_O and the camera coordinate system R_C. Next, the reconstruction step consists in placing, rotating, scaling, and translating a rectangular 3D box that must fit at best with the potential objects within the scene as seen through the single image. With each face of a rectangular box, a texture that may contain holes due to invisible parts of certain objects is assigned. We show how the textures are extracted and how these holes are located and filled. Our method has been applied to various real images (pictures scanned from books, photographs) and synthetic images.

Key words: Calibration – Reconstruction – Rendering – Texture extraction

Correspondence to: E. Guillou

1 Introduction and motivation

Modeling the geometry and the photometry (reflectance, Bidirectional Reflectance Distribution Function (BRDF), transmittance, light source intensity) of a real scene, for a purpose of a walkthrough or augmented reality, is known to be a very difficult task. Indeed, a real scene contains objects of highly detailed geometry and of complex physical properties. In addition, the light-material interactions (scattering in a participating medium, glossy reflection, transmission through rough glass, spectral nature of light, diffraction, etc.) still are difficult to simulate.

One solution to this problem is to make use of image-based rendering methods (IBR) [1–6]. These methods need the acquisition of a large number of images (for example, for building mosaics). This is why image-based modeling techniques (IBM) [7–10] are preferred to IBR methods. These techniques recover, from a set of real images, the geometry of either polygonal objects or geometric primitives with known models. To improve the realism, textures are extracted and may or may not be view dependent.

For most IBM techniques, camera calibration is needed. This operation is often performed by inserting a calibration target in the real scene. The literature that has addressed this problem is impressive, as will be seen later on.

In this paper, we show how camera calibration and 3D reconstruction can be achieved from a single image, unlike the existing IBR and IBM techniques. The assumptions made by our method are: the single real image contains two vanishing points, one line segment (for determing the translation vector) in the image is known, the principle point is the center of the image, and the aspect ratio is fixed by the user. No calibration target is needed, which makes the method more general. The calibration of the camera consists in computing the focal length and the rotation and translation matrices with respect to a coordinate system defined by the vanishing points. Once the camera has been calibrated, the reconstruction algorithm is invoked. This algorithm tries to bound objects in the image with 3D rectangular boxes that can be rotated, scaled, and translated. Each visible face of a box is assigned an extracted texture representing the appearance of the objects through its associated box. Note that the reconstruction in question is coarse, but it is sufficient for our targeted applications. Next, VRML and PovRay descriptions of the reconstructed scene are generated so that we can walk through it or

© Springer-Verlag 2000

generate images from novel viewpoints. Note that, hitherto, the box faces, not visible in the input image, have not been assigned any texture. We fill these holes with image patterns picked up from the image.

The rest of this paper is organized as follows. Section 2 outlines the contents of the existing papers related to our work. The calibration process is detailed in Sect. 3, while the 3D reconstruction step is described in Sect. 4. Before concluding, implementation details and results are given in Sect. 5.

2 Prior works

For various computer vision applications, it is important to calibrate cameras [11]. Calibrating a camera consists in determining its intrinsic parameters (focal length, pixel ratio, center of projection) and its extrinsic parameters (rotation and translation expressing the position and orientation of the camera with respect to the world coordinate system). There are many existing techniques for solving this problem, among which very efficient calibration techniques have been proposed by Tsai [12] and Faugeras and Toscani [13]. Other techniques exploiting the presence of vanishing points in the image have been devised. We call these techniques *vanishing-point-based calibration* (VPBC) techniques. In this section, we briefly summarize only the VPBC techniques since they are related to our work. We also outline the contents of four papers [14–18] dealing with 3D reconstruction from a single picture or a single panoramic image mosaic.

2.1 VPBC Methods for camera calibration

Haralick's method [19] does not explicitly make use of vanishing points. It uses a similar idea – the presence of a rectangle in the scene with a perspective projection that is visible on the image plane. His method makes the following assumptions: the focal length is known, the center of projection is located at the origin of the world coordinate system (WCS), and the camera performs only rotations. The rotation of the camera is decomposed into three rotations about each axis of the WCS.

In their paper [20], Caprile and Torre calibrate a camera from two or more images. In a first step, the focal length and the location of the intersection between the optical axis and the image plane are recovered from a single image of a cube (calibration target). In a second step, the rotation matrix and the translation vector that describe the rigid motion between the coordinate systems fixed in the two cameras are estimated from an image stereo pair of a suitable planar pattern (calibrating pattern). By matching the corresponding vanishing points (three in this case) in the two images, we can compute the rotation matrix and estimate the translation vector by means of a simple triangulation.

Shigang et al. [21] propose a calibration method applied to a robot with a vision sensor moving around an indoor environment. The following assumptions are made: (1) the robot moves on a flat horizontal plane, and so does the camera, the axis of which is kept horizontal and perpendicular to the camera velocity; (2) the environment contains many straight lines; and (3) the camera velocity and the focal length are known. The method first extracts horizontal lines from a sequence of images and then estimates the camera rotation from the motion of the corresponding vanishing points.

Chen and Jiang [22] make only one assumption: the focal length is known. They put a calibrating pattern (which is a uniform grid) in the environment and determine the lines defined by this pattern. Once these lines have been extracted, they classify them into two groups: horizontal and vertical lines. From the vanishing points associated with these groups of lines, and with the assumption that two line directions have been marked on the grid pattern (corresponding to the x and y axes of the WCS), the rotation matrix is determined. Then the translation vector is computed from the location of the lines of the grid pattern.

In Wang and Tsai's work [23], a hexahedron is employed as a calibration target to generate a vanishing line of the ground plane from its projected image. The vanishing line includes useful geometric hints about the camera orientation parameters and the focal length, from which the orientation parameters can be computed analytically. As for the camera position parameters, they can be recovered with the aid of geometric projective relationships.

In a more recent paper, Liebowitz et al. [17] make use of three vanishing points determined from three sets of parallel lines available in architectural scenes. The camera is calibrated from the rectification parameters of a set of planes. Their method does not need to estimate the rotation and translation parameters of the camera.

2.2 3D Reconstruction from a single image or a single panoramic image mosaic

To our knowledge, four papers address the problem of 3D reconstruction from a single image [14, 16–18] and one from a single panoramic image mosaic [15]. The method, described in [14], does not employ any conventional computer vision technique, but makes use of a spidery mesh to obtain a very simple scene model from a single image using a user interface. The aim is to provide a new type of visual effects for various animations rather than to recover a 3D model for a walkthrough. The method starts by specifying one vanishing point in the image. Then the geometry of the background is approximated with a model composed of five rectangles. This model is a polyhedronlike form with the vanishing point on its base. Each foreground object (near the viewer) is modeled with hierarchical polygons. The billboardlike representation is used for the foreground objects.

Parodi and Piccioli [16] have investigated the reconstruction of 3D models in a scene from line drawing (on a single image). Geometrical constraints are used, and they are provided by the location of vanishing points and an incidence structure I represented by a graph with nodes that are vertices and planar panels. A spatial structure $S = (I, T)$ is also determined, where T is a set of depth relations. The connected components of this graph are determined. A component can be seen as a different object or a set of physically connected objects. Given the location of a 3D vertex, the authors propose a formula that determines the 3D coordinates of other vertices belonging to the same planar panel. This formula contains some terms depending on two vanishing points determined from the planar panel. The method computes the vanishing points associated with each planar panel. The focal length is calculated with the algorithm described in [20]. A scale factor is associated with each component. To check the realizability of the reconstruction, three independent tests are performed in sequence. In addition, one of these tests checks if there is a set of scale factors such that the resulting 3D locations of all vertices simultaneously satisfy the incidence and depth relations.

In [15], Shum et al. present an interactive modeling system that constructs a 3D model from one or several panoramic image mosaics. The system recovers the camera pose for each mosaic from known line directions and points, and then constructs the 3D model using all available geometrical constraints (soft or hard). The modeling process is formulated as a linearly constrained least-squares problem which can be solved with the QR factorization method. The method makes use of parallel lines and/or known points to recover the camera rotation. For a single panorama, the translation is set to zero if no point in the 3D model is given. Textures are also extracted by the method in [24].

Liebowitz et al. [17] propose a two-pass reconstruction method applied to planar façades. In a first step, the visible façades are individually reconstructed by rectification. The second step proceeds in an incremental fashion, that is to say, the façades are positioned in turn. More precisely, a façade A can be positioned relatively to another facade B only if A and B share common points. The orientation of the facade B, as well as its size, is determined with the common points and the rectification parameters of the planes supporting the two façades A and B.

Van Den Heuvel [18] describes a method for reconstructing polyhedral object models from a single image. The method uses measurements of image lines, object information, etc. Topology (of the coplanarity type) and geometry (parallelism, perpendicularity, symmetry) constraints are used by the reconstruction process. The method assumes that the camera is calibrated.

2.3 Discussion

The VPBC methods described have the advantages of requiring only a few images and little computing time. However, either they make a certain number of assumptions or require placing in the scene a calibration target or a calibrating pattern. These methods are unusable if the user has images to be calibrated, but is unable to access the real 3D scene for including a calibrating pattern. To remedy this, as in [15–17], the calibration method described in this paper needs input data consisting of only one image (a picture) containing two vanishing points instead of three, as required by most of the existing VPBC techniques. It does not need a calibrating pattern or a target.

Recall that [14] provides a means for extracting an approximated 3D model of the scene background and for representing the foreground objects with billboards from a single image. By means of this, we can make a tour into the picture without the need of camera calibration or actual 3D reconstruction. With this technique, the set of allowed trajectories is limited.

In [16], vanishing points are computed for each planar panel; incidence and spatial structures are determined, and the realizability of the recovered models is checked by three independent tests. We think that these processes require much computing time, which limits interactivity (this paper does not give any result on the computing time). Unlike this method, our approach needs only two vanishing points for the whole scene, and it does not require any realizability check since our 3D model is coarse (a rectangular box). Moreover, we do not need to determine the incidence and spatial structures, but only to draw two sets of parallel lines on the single image.

In [15, 16], to avoid any ambiguity on the determination of the rotation matrix, the authors suggest selecting points of known coordinates. For recovering a model with six faces, the system takes 15 min. For recovering a model of moderate complexity, we think that much more modeling time would be needed. Unlike this method, our approach allows us to recover, without ambiguity, the rotation matrix using only two sets of parallel lines, each set defining a vanishing point. Furthermore, our approach operates on a single image, not a panoramic image mosaic. However, their reconstruction method is more precise than ours.

In [18], the reconstruction process relies on the extraction of straight lines from which line segments (corresponding to edges of objects) are selected. Topological relations between the line segments have to be specified by the user. These relations enable us to close polylines corresponding to the border of the face of an object. A set of constraint equations is determined and solved to help reconstruct the objects as seen through the single image. This method seems promising, but it assumes that the camera is calibrated (intrinsic parameters).

Although it was done independently, the most recent work described in [17] and our work have several similarities: extraction of vanishing points from sets of parallel lines on the single image, no calibration target, reconstruction from a single image. However, some differences exist. The first one concerns camera calibration. Indeed, unlike [17], our method determines the camera pose, say, the translation vector and the rotation matrix. With this, we can reconstruct each primitive object independently of the others. Furthermore, our reconstruction method is not limited to façades.

We want to propose a method for calibrating the camera precisely and for recovering the geometry (coarsely) and the photometry (by means of extracted textures). This method must provide an easy walkthrough (with a large set of possible trajectories) in the reconstructed 3D model rather than in the picture. Unlike [17], where some skills about computer vision and projective geometry are required as background, our main goal is to propose a simple and fast reconstruction system that is within the capability of any researcher in computer graphics who has no experience in the field of computer vision.

3 Calibration using two vanishing points

We now make the following assumptions. The image contains two vanishing points; the principal point is the center of the single image, and the aspect ratio is fixed by the user. We will see that the translation vector is determined up to a scale factor if we do not know the length of one line segment in the 3D space. In this section, we give details about several points of our calibration method to make this paper clear and understandable for a computer-graphics reader with no experience in computer vision. These details can be skipped by a computer-vision reader.

3.1 Vanishing point, line, and plane

Let D be a line in the 3D space passing through a point $Q_0 = (a_x, a_y, a_z)^t$ and of direction $d = (d_x, d_y, d_z)^t$. Its parametric equation is: $Q = Q_0 + l \cdot d$, where l is a parameter and $Q = (x, y, z)^t$.

The vanishing point associated with D is the point $F_D = (x_\infty, y_\infty, z_\infty)^t$ on the image plane. Its coordinates are:

$$\begin{cases} x_\infty = \lim_{l\to\infty} f\frac{a_x+l.d_x}{a_z+l.d_z} = f\frac{d_x}{d_z} \\ y_\infty = \lim_{l\to\infty} f\frac{a_y+l.d_y}{a_z+l.d_z} = f\frac{d_y}{d_z} \\ z_\infty = f \end{cases} . \quad (1)$$

All the lines parallel to D define the same vanishing point F_D. The vanishing line L_v associated with a plane Π contains the vanishing points defined by all the lines supported by this plane. From now on, the plane passing through the center of projection of the camera and containing L_v is called the vanishing plane associated with Π.

3.2 Determining the focal length from a single image

We consider a pinhole camera model. The center of projection is O, and its projection onto the image plane is P. Suppose the image contains two sets of parallel lines defining two vanishing points. Suppose the lines of the first set are perpendicular to those of the second. Take two lines in each set so that their intersection is a rectangle $ABCD$ in the 3D space. Let us call $abcd$ its perspective projection (see Fig. 1). Let $\boldsymbol{u}$ and $\boldsymbol{v}$ be the direction vectors of the lines (AD) and (AB), respectively. Recall that $(AD) \parallel (BC)$ and $(AB) \parallel (DC)$. We suppose that the rectangle $ABCD$ is not parallel to the image plane. The lines (AD) and (BC) define the same vanishing point $F_u = (ad) \bigcap (bc)$ on the image plane. In the same way, (AB) and (DC) have a vanishing point $F_v = (ab) \bigcap (dc)$. We suppose the coordinates of F_u and F_v, expressed in the camera coordinate system, are known.

Let $\Pi(O, \overrightarrow{OF_u}, \overrightarrow{OF_v})$ be the vanishing plane associated with $ABCD$. The vanishing line defined by $ABCD$ is the line $(F_u F_v)$, which is also the intersection between $\Pi(O, \overrightarrow{OF_u}, \overrightarrow{OF_v})$ and the image plane. Let P_{uv} be the orthogonal projection of P on the the line $(F_u F_v)$. We want to compute the focal length:

$$f = OP = \sqrt{{OP_{uv}}^2 - {PP_{uv}}^2}.$$

Since the coordinates (in the camera coordinate system) of P, F_u and F_v and the length of PP_{uv} are known, only the length of OP_{uv} has to be calculated for evaluating the focal length.

We suppose the triangles OF_uF_v, OF_uP_{uv}, and OF_vP_{uv} are right angled, with $\hat{O}$, $\hat{P_{uv}}$, $\hat{P_{uv}}$ as the right angles, respectively (see Fig. 2). We then have

$$\begin{cases} \alpha + \gamma &= \frac{\pi}{2} \\ \beta + \theta &= \frac{\pi}{2} \\ \gamma + \theta &= \frac{\pi}{2} \end{cases}. \tag{2}$$

From (2) we derive:

$$\begin{cases} \alpha &= \theta \\ \gamma &= \beta \end{cases}. \tag{3}$$

Consequently, as $\alpha = \theta$ and the triangles OF_uP_{uv} and OF_vP_{uv} are right angled with $\hat{P_{uv}}$ as the right angle, then:

$$\frac{OP_{uv}}{P_{uv}F_u} = \frac{F_vP_{uv}}{OP_{uv}}. \tag{4}$$

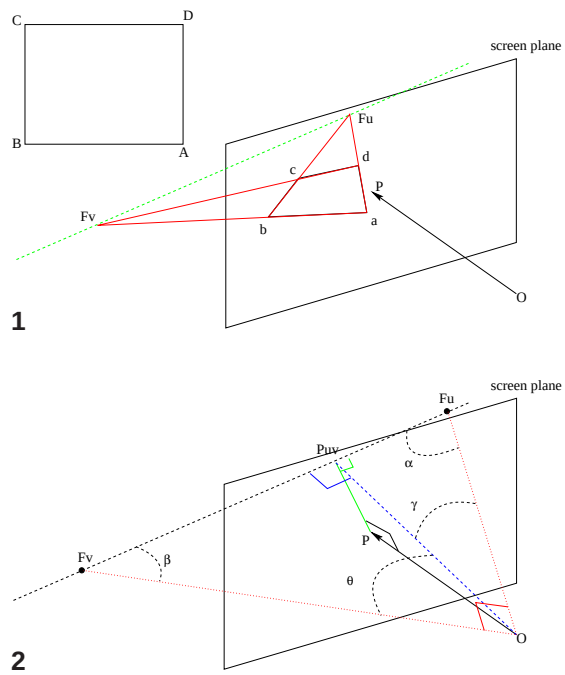


Fig. 1. Focal length

Fig. 2. Focal length

We then have:

$$OP_{uv} = \sqrt{F_vP_{uv} * P_{uv}F_u}. \tag{5}$$

Let us prove now that the triangles OF_uF_v, OF_uP_{uv}, and OF_vP_{uv} are right angled. Since F_u (or F_v) is the vanishing point associated with the direction $\boldsymbol{u}$ (or $\boldsymbol{v}$), then $\overrightarrow{OF_u} = k_u\boldsymbol{u}$ (or $\overrightarrow{OF_v} = k_v\boldsymbol{v}$). By definition, $\boldsymbol{u} \cdot \boldsymbol{v} = 0$; thus, $\overrightarrow{OF_u} \cdot \overrightarrow{OF_v} = 0$. Consequently $(OF_u) \perp (OF_v)$, which proves that the triangle OF_uF_v is right angled with $\hat{O}$ as the right angle.

By construction, $\overrightarrow{OP} \cdot \overrightarrow{F_uF_v} = 0$ and $\overrightarrow{PP_{uv}} \cdot \overrightarrow{F_uF_v} = 0$. We can write $\overrightarrow{(OP_{uv})} \cdot \overrightarrow{F_uF_v} = \overrightarrow{OP} \cdot \overrightarrow{F_uF_v} + \overrightarrow{PP_{uv}} \cdot \overrightarrow{F_uF_v} = 0$; hence, the lines (OP_{uv}) and (F_uF_v) are perpendicular. The triangles OF_uP_{uv} and OF_vP_{uv} are then right angled with $\hat{P_{uv}}$ as the right angle.

3.3 Computing the rotation matrix

The problem now is to compute the rotation matrix that describes the rigid motion between the world and camera coordinate systems, which are called

R_o and R_c respectively. R_o may have any point in the 3D scene (the point A for example) as an origin. It is defined by three basic vectors $\boldsymbol{u}$, $\boldsymbol{v}$ (see Sect. 3.2) and $\boldsymbol{w}$ (such that $\boldsymbol{w} = \boldsymbol{u} \times \boldsymbol{v}$). See Fig. 3 for further details. We can write $R_o = (A, \boldsymbol{u}, \boldsymbol{v}, \boldsymbol{w})$ and $R_c = (O, \boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k})$. The 2D coordinate system associated with the screen plane can be denoted by $R_s = (P, \boldsymbol{i}, \boldsymbol{j})$, P being the orthogonal projection of the center of projection of the camera onto the screen plane.
We adopt the following notation: $\boldsymbol{V}_{/R}$ means that the vector $\boldsymbol{V}$ is expressed in the coordinate system R. The objective is to determine the rotation matrix $M_{o\to c}$ so that:

$$M_{o\to c} \bullet \boldsymbol{u}_{/R_o} = \boldsymbol{u}_{/R_c}\,,\ M_{o\to c} \bullet \boldsymbol{v}_{/R_o} = \boldsymbol{v}_{/R_c} \text{ and } M_{o\to c} \bullet \boldsymbol{w}_{/R_o} = \boldsymbol{w}_{/R_c}\,. \tag{6}$$

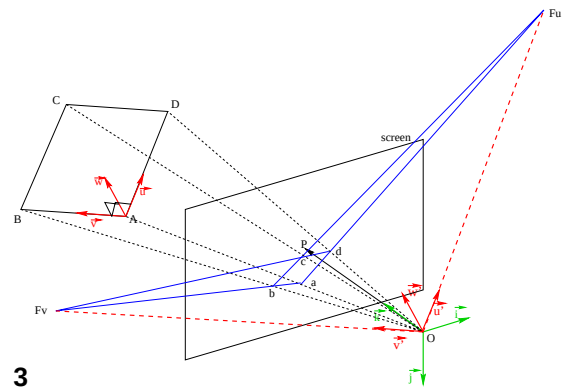
Let $D_{u'}$ and $D_{v'}$ be the lines that pass through the center of projection O and that are aligned with the vectors $\boldsymbol{u}'$ and $\boldsymbol{v}'$, respectively (Fig. 3). Their respective vanishing points are then F_u and F_v. This proves that the basic vectors $(\boldsymbol{u}, \boldsymbol{v}, \boldsymbol{w})$ and $(\boldsymbol{u}', \boldsymbol{v}', \boldsymbol{w}')$ are similar. Consequently, the same rotation describes the rigid motion between $R_c(O, \boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k})$ and $R_o(A, \boldsymbol{u}, \boldsymbol{v}, \boldsymbol{w})$, and between $R_c(O, \boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k})$ and $R_{o'}(O, \boldsymbol{u}', \boldsymbol{v}', \boldsymbol{w}')$. The coordinates of the vectors $\boldsymbol{u}'$, $\boldsymbol{v}'$ and $\boldsymbol{w}'$ in R_c are known:

$$\begin{aligned}
\boldsymbol{u}'_{/R_c} &= \frac{\overrightarrow{OF_u}_{/R_c}}{\|\overrightarrow{OF_u}_{/R_c}\|} = \left(\frac{F_{u_i}}{s_1}, \frac{F_{u_j}}{s_1}, \frac{f}{s_1}\right)^t;\\
\boldsymbol{v}'_{/R_c} &= \frac{\overrightarrow{OF_v}_{/R_c}}{\|\overrightarrow{OF_v}_{/R_c}\|} = \left(\frac{F_{v_i}}{s2}, \frac{F_{v_j}}{s2}, \frac{f}{s2}\right)^t;\\
\boldsymbol{w}'_{/R_c} &= \boldsymbol{u}'_{/R_c} \times \boldsymbol{v}'_{/R_c},
\end{aligned} \tag{7}$$

with $\boldsymbol{w}'_{/R_c} = (w'_i, w'_j, w'_k)$, $s1 = \|\overrightarrow{OF_u}_{/R_c}\|$, $s2 = \|\overrightarrow{OF_v}_{/R_c}\|$ and f is the focal length.
We can write:

$$M_{o\to c} \bullet \boldsymbol{u}_{/R_o} = \boldsymbol{u}'_{/R_c}\,,\ M_{o\to c} \bullet \boldsymbol{v}_{/R_o} = \boldsymbol{v}'_{/R_c} \text{ and } M_{o\to c} \bullet \boldsymbol{w}_{/R_o} = \boldsymbol{w}'_{/R_c}\,. \tag{8}$$

Knowing that $\boldsymbol{u}_{/R_o} = (1, 0, 0)$, $\boldsymbol{v}_{/R_o} = (0, 1, 0)$, $\boldsymbol{w}_{/R_o} = (0, 0, 1)$ and using (7) and (8), we get:



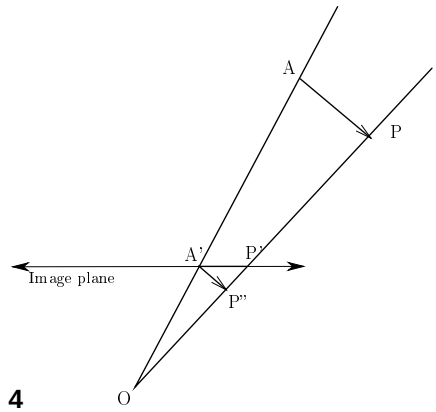


Fig. 3. Determing the rotation matrix
Fig. 4. Determining the translation vector

$$M_{o\to c} = \begin{pmatrix} \frac{F_{u_i}}{\sqrt{{F_{u_i}}^2+{F_{u_j}}^2+f^2}} & \frac{F_{v_i}}{\sqrt{{F_{v_i}}^2+{F_{v_j}}^2+f^2}} & w'_i \\ \frac{F_{u_j}}{\sqrt{{F_{u_i}}^2+{F_{u_j}}^2+f^2}} & \frac{F_{v_j}}{\sqrt{{F_{v_i}}^2+{F_{v_j}}^2+f^2}} & w'_j \\ \frac{f}{\sqrt{{F_{u_i}}^2+{F_{u_j}}^2+f^2}} & \frac{f}{\sqrt{{F_{v_i}}^2+{F_{v_j}}^2+f^2}} & w'_k \end{pmatrix}. \tag{9}$$

3.4 Computing the translation vector

Consider Fig. 4. Let A' be the perspective projection of A, and $\overrightarrow{A'P'}$, that of the vector $\overrightarrow{AP}$, which is a vector parallel to the axis $\boldsymbol{u}$ of R_o and originating at A, the origin of R_o. To determine the translation vector, we suppose that the length l of $\overrightarrow{AP}$ is known; otherwise, the translation vector will be computed up to a scale factor.
We can write:

$\overrightarrow{AP}_{/R_o} = l \cdot \boldsymbol{u}$; $\overrightarrow{AP}_{/R_c} = M_{o\to c}.\overrightarrow{AP}_{/R_o}$.

Let P'' be the intersection of the line (OP) with the line D passing through A' and whose direction vector is $\overrightarrow{AP}$. Then:

$$P''_{/R_c} = (OP)_{/R_c} \cap D = (OP')_{/R_c} \cap D. \tag{10}$$

As the triangles $OA'P''$ and OAP are similar, from Thales' theorem we get:

$$\frac{\|\overrightarrow{A'P''}_{/R_c}\|}{\|\overrightarrow{AP}_{/R_c}\|} = \frac{\|\overrightarrow{OA'}_{/R_c}\|}{\|\overrightarrow{OA}_{/R_c}\|} \tag{11}$$

or

$$\|\overrightarrow{OA}_{/R_c}\| = \frac{\|\overrightarrow{OA'}_{/R_c}\|.\|\overrightarrow{AP}_{/R_c}\|}{\|\overrightarrow{A'P''}_{/R_c}\|}; \tag{12}$$

hence:

$$\overrightarrow{OA}_{/R_c} = \|\overrightarrow{OA}_{/R_c}\|.\frac{\overrightarrow{OA'}_{/R_c}}{\|\overrightarrow{OA'}_{/R_c}\|}. \tag{13}$$

The translation vector is:

$$\boldsymbol{T}_{o\to c/R_o} = M_{c\to o}.\overrightarrow{OA}_{/R_c}. \tag{14}$$

3.5 Discussion

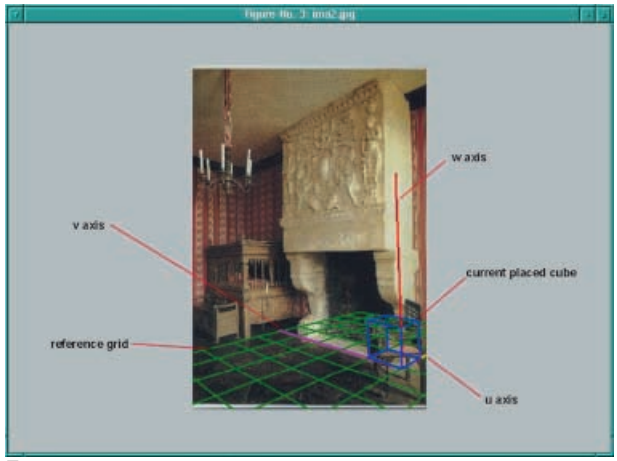
As the vanishing points are determined manually, one has to compute them with precision. For this reason, the resolution of the single image used must be high. Note that vanishing points can be extracted automatically [25, 26], and our method is not limited to pose estimation since it determines the focal length. Moreover, as we are interested in coarse 3D reconstruction for walkthrough, we can neglect camera distortions and assume that the principle point is close to the image center.

4 Coarse 3D reconstruction

4.1 Principle

Once the calibration step has been completed, the user can start the reconstruction process from the single image. It operates as follows.

First we generate a planar reference grid (with a fine resolution) supported by the uv plane of the WCS R_o. As the camera has been calibrated, and R_o has been determined, a unit cube is placed in the scene and displayed on the image (by composition). The lower left vertex of its front facing face is placed at the origin of R_o. This cube is moved, scaled, and rotated with respect to the u, v, and w axes so as to bound an object in the scene. The result is a parallelepiped that fits as well as possible with the object. These geometric transformations are facilitated thanks to the grid. See Fig. 5 for further details. This operation results in a set of rectangular boxes, each one bounding an object as seen in Fig. 6. Each face of a cube is assigned a texture corresponding to the image of the associated object as seen by the viewer (center of projection). Note that a cube's face may partially be occluded by the faces of other objects, which generates holes in the textures. These holes are filled with other textures extracted from the image.



5

6

Fig. 5. The grid (*green*) and a box bounding a chair (*blue*)
Fig. 6. All the extracted 3D boxes in various colors

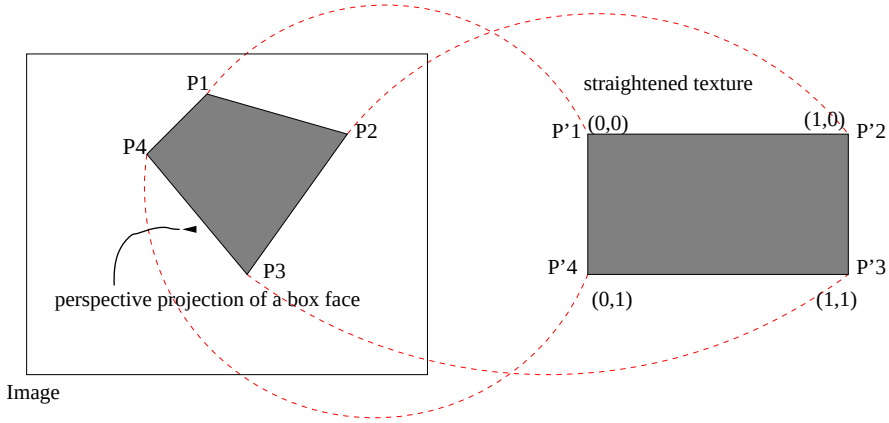


Fig. 7. Texture extraction and rectification: the texture coordinates range from 0 to 1

4.2 Placing the unit cube

For each potential object, the planar reference grid is placed on the uv plane of the WCS R_o thanks to a dialog box (generated by our user graphics interface) that allows the user to enter five parameters: the grid resolution and its size given by $u_{\min}$, $u_{\max}$, $v_{\min}$, and $v_{\max}$. Then this grid is placed just under the object to be reconstructed by moving it along the w axis with two keys. The coordinates of the grid points are displayed onto the image. By using another dialog box, the user can translate, rotate, and scale the unit cube (originally placed in such a way the lower left corner of its front face coincides with the origin of R_o) to fit to the object. Note that objects such as ceilings and floors are bounded with rectangular boxes of very small thickness.

4.3 Texture extraction

A face of a rectangular box is projected onto the image plane. The result is a quadrilateral to which a rectangular texture is assigned, as explained in Fig. 7. Let x' and y' be the texture coordinates $x' \in [0, 1]$ and $y' \in [0, 1]$. The 2D projective transformation $H = (h_{ij})$ mapping a point $P' = (x', y', 1)^t$ of the texture to a point $P = (x, y, 1)^t$ of the image is computed as follows (see Fig. 7 for the notations). Each vertex $P'_i = (x'_i, y'_i, 1)^t$ of the texture is transformed to a vertex $P_i = (x_i, y_i, 1)^t$ of the projected box's face as: $P_i = H \bullet P'_i$. More precisely [4]:

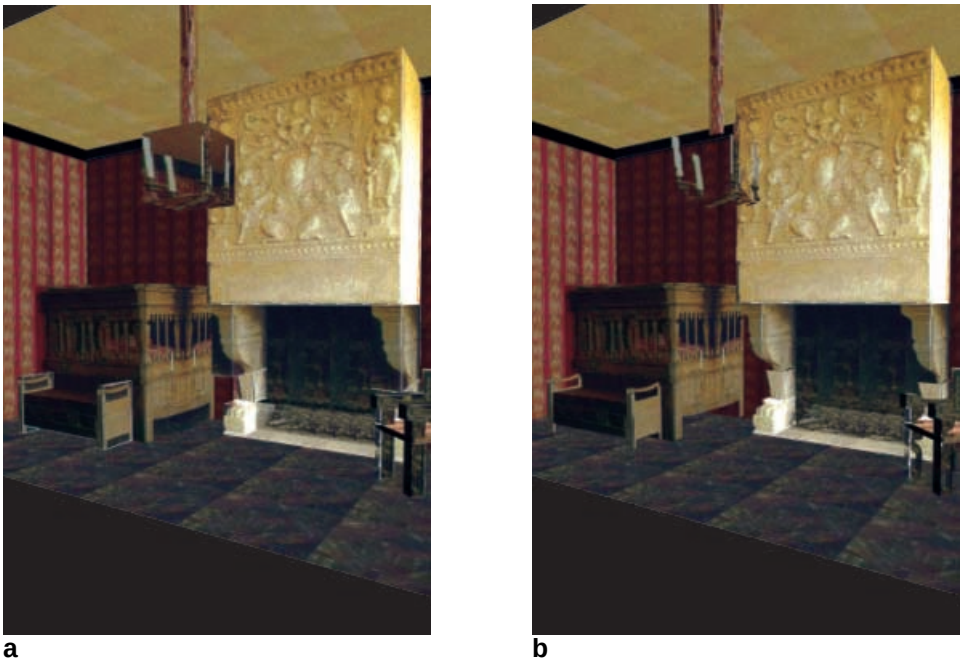
$$x_i = \frac{h_0 x'_i + h_1 y'_i + h_2}{h_6 x'_i + h_7 y'_i + 1}, \quad y_i = \frac{h_3 x'_i + h_4 y'_i + h_5}{h_6 x'_i + h_7 y'_i + 1} \tag{15}$$

with

$$H = \begin{pmatrix} h_0 & h_1 & h_2 \\ h_3 & h_4 & h_5 \\ h_6 & h_7 & 1 \end{pmatrix}. \tag{16}$$

Note that this mapping operates on homogeneous coordinates, (x, y) and (x', y') being cartesian coordinates. We get a system of equations with the h_js as unknowns. Next, each texel of the texture is mapped to a pixel of the image (with the aid of H) and is assigned the color of this pixel. The result is a rectified rectangular texture. To avoid mapping several pixels to the same pixel of the image, the resolution (in pixels) of this texture is chosen as the number of pixels covered by the smallest edge of the projected box face for each coordinate axis of the texture. For example, in the case of Fig. 7, these edges are P_1P_4 (since $P_1P_4 < P_2P_3$) and P_1P_2 ($P_1P_2 < P_4P_3$). The resolution is then the number of pixels covered by each of these edges.

Note that the extracted texture of each box's face depends on the single view. Thus it is view dependent. During a walkthrough of the reconstructed 3D model, the texture mapping process uses only this single texture, even though the camera changes position and orientation. This is due to the fact that only one image is available. This situation can cause the following problem. Let us take the chandelier in Fig. 8 as an example. As the faces of the box associated with the chandelier are opaque, certain pieces



a **b**

Fig. 8a, b. The reconstructed 3D model as seen from a novel viewpoint: **a** the textures associated with the chandelier are opaque; **b** the textures have been modified to make them partially transparent

of the background that should be visible from a novel viewpoint are occluded by these faces. To remedy this, the textures associated with the faces visible from the original viewpoint are modified. The portions of the textures that do not correspond to the chandelier are made transparent (our graphics user interface is used for this) with the help of the α channel. The obtained result is shown in Fig. 8.

If the user has several photographs of the same scene taken from different viewpoints, the faces of all the boxes will be assigned several view-dependent textures, one for each viewpoint. To render the 3D model from a novel point of view, texture mapping would be performed with a weighted combination of the extracted textures as explained in [7]. Another solution is to extract view-independent textures from multiple photographs [9].

4.4 Detecting and filling the holes

The holes correspond to the portions of the box faces that are invisible from the viewpoint associated with the single image. The hole-filling problem has already been addressed in [7, 8, 27, 28]. Since all the proposed algorithms make use of multiple images, they cannot be applied to our case (a single image). This is why we have implemented our own method, which we describe hereafter. Consider Fig. 9 to explain how holes in the textures are determined and filled. Let A and B be two boxes. To see whether B occludes A or not, each box is projected onto the screen plane. The result is two polygons on the screen. Using a min-max technique, we compute a rectangular bounding box aligned with the screen axes for each of these two polygons. If the two resulting bounding boxes intersect, then a face of B could occlude a face of A. In this case, to see if a face F_A of A is occluded by a face F_B of B, we project these two faces and compute their intersection I on the screen plane. If this intersection is not empty, then we check if F_B occludes F_A by tracing a few rays originating at the viewpoint and passing through I. In the case of occlusion, I is an occlusion polygon corresponding to a hole in the texture assigned to the face F_A. This hole-detection process is repeated for all the faces of B and all the other potential occluding

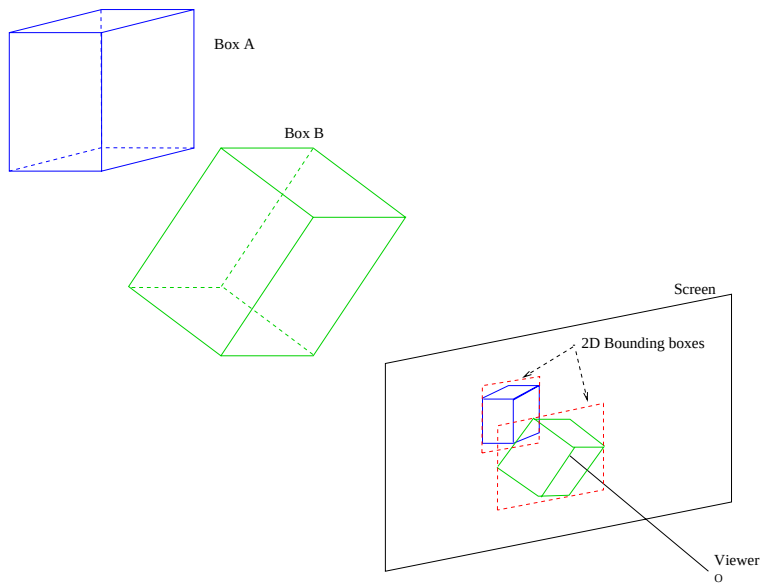


Fig. 9. Object *B* occludes object *A*

boxes. For each box face, the result is a set of occlusion polygons lying on the screen. To fill these holes, our method proceeds as follows. Let F be a face containing holes. We project F onto the screen and scan-convert the associated occlusion polygons in black. Then we extract one or several appropriate image patterns from the initial image (using our graphics user interface), and we use these patterns to fill the black regions that correspond to the holes. Next, we construct the new texture assigned to F, as already explained.

Let us consider Fig. 5 and, more precisely, the chimney to show how image patterns are extracted for filling the holes. In this figure (the single image), we see only the front-facing part and the right side of the chimney, while the left side is occluded. When a box is placed around the chimney, the left side of the chimney corresponds to a large hole that we can fill with the visible texture associated with the right face of the chimney. We suppose, in this case, that the right and left faces of the chimney are similar.

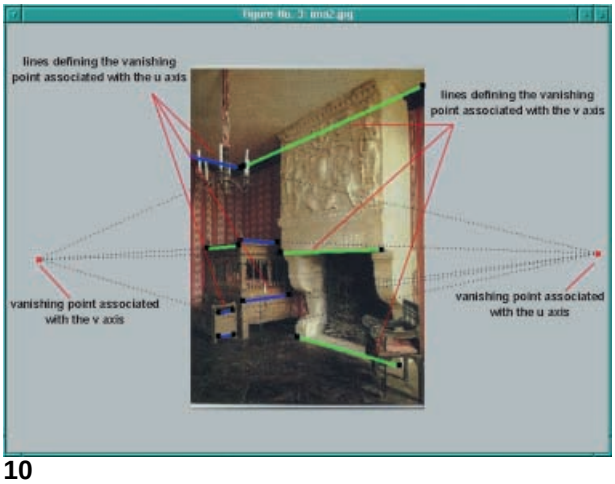
4.5 Discussion

At present, our reconstruction method does not handle cast shadows precisely. This due to our technique of filling holes with image patterns extracted from the image. Trees and any vegetation can be coarsely reconstructed, which makes our method applicable to any kind of scene: indoor and outdoor.

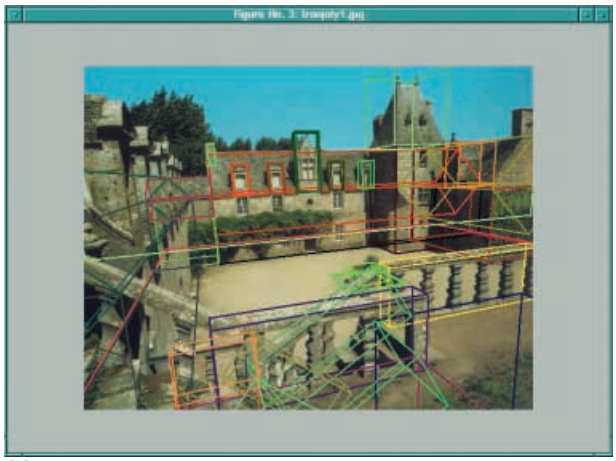
5 Results

Our method has been applied to various real and computer generated images. Let us look at some results obtained for the image of Figs. 5, 6, and 10. This image has been scanned from a book. Its resolution is 960×612. Calibration and reconstruction took less than 1 h, and 30 boxes were determined to bound the potential objects. Our system was implemented with Matlab. We expect a shorter reconstruction time with other programming languages such as C or C++. The calibration step is very fast. Placing boxes around the potential objects is a simple and fast operation. However, the texture extraction step is computationally more expensive than the other steps (more than 60% of the total modeling time).

Figure 10 shows the two sets of lines (blue and green), giving the two vanishing points needed by the calibration and the reconstruction steps. For each vanishing point, the user selects (with the mouse) a set of potential parallel lines. To make this selection easier, the user can zoom in on the portions of



10



11

12

Fig. 10. Determining the vanishing points

Fig. 11. All the extracted 3D boxes in various colors

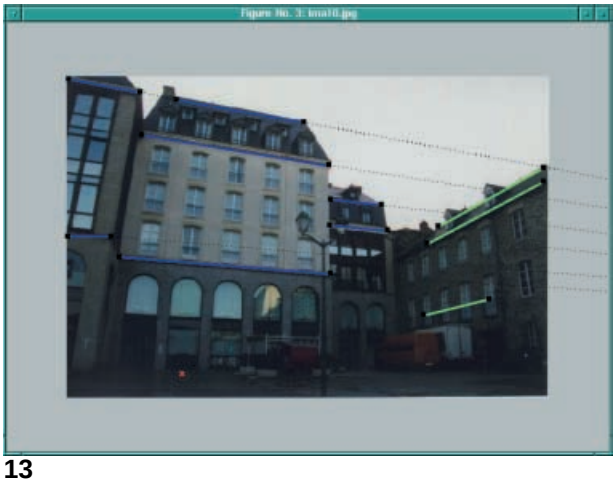
Fig. 12. Reconstructed 3D model as seen from a novel viewpoint

the image containing these lines. We prefer to select a maximum number of lines for a set, for more precision. Note that vanishing points can be extracted automatically with the method described in [25, 26]. Once the vanishing points have been determined, we display a 2D grid on the image to guide the user in placing a 3D box around an object, as seen in Fig. 5. In this figure, we see a blue box bounding a chair. This process is repeated for all the potential objects seen in the image. The result is a set of boxes as those shown in Fig. 6. In this figure, the boxes have been displayed with various colors for reasons of clarity. Afterwards, textures are extracted and assigned to the faces of all the boxes. Next, the user selects and extracts appropriate portions of the image that will be used for filling the holes in the faces' textures.

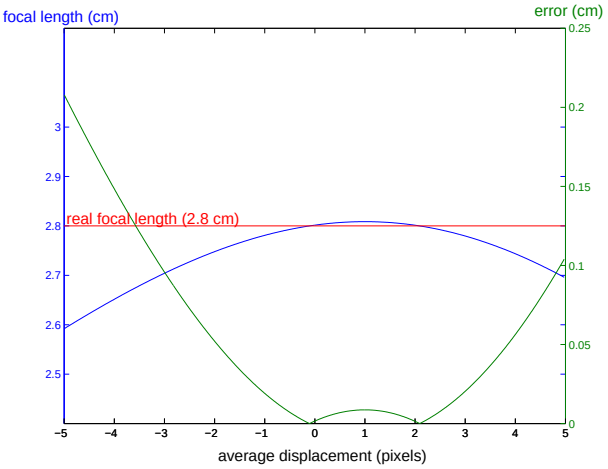
The extracted 3D model is converted into the VRML and PovRay formats, and finally saved in two files. These files facilitate an easy walkthrough (with a VRML browser) and the generation of other images from various viewpoints (Fig. 8).

The image given in Fig. 11, acquired with a classical camera, has a resolution of 392×512. It required 31 boxes to bound the potential objects. Calibration and reconstruction took less than 2 h. Figure 12 is an image of the reconstructed model as seen from a novel viewpoint.

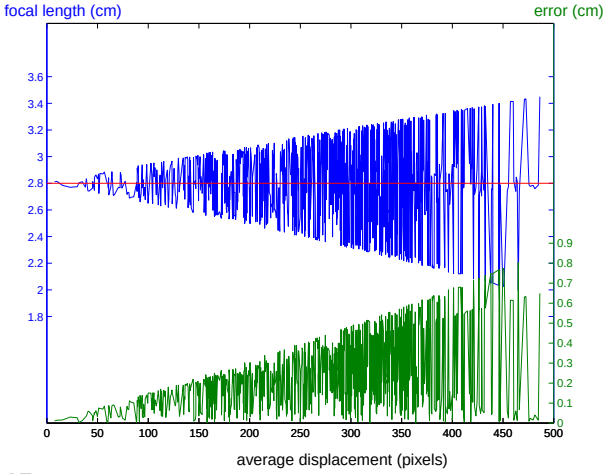
Recall that our camera calibration method relies on vanishing points determined by manually selecting lines on the single image. This manual selection may affect the precision of the recovered camera parameters and the image coordinates. Measuring this precision amounts to comparing the actual values of the parameters with the recovered ones. Unfortunately, only the actual value of the focal length may be available. Let us now see how we can assess the precision



13



14



15

Fig. 13. Photograph with known focal length. The two sets of parallel lines are displayed in *blue* and *green*

Fig. 14. Error on focal length estimation: displacement of parallel lines

Fig. 15. Error on focal length estimation: displacement of one vanishing point

of the recovered focal length in the case of image of Fig. 13. The actual focal length of this image is 2.8 cm. We first manually determine a set of parallel lines (defining a vanishing point) with a maximum of precision by zooming in on the single image. We find a focal length equal to 2.8051 cm. Let us now simulate errors in the process of selecting parallel lines. Each of these lines is represented by a ray (of the same direction) originating at any point lying on this line (for instance the leftmost point). Next, for each ray, its origin is randomly moved away from its initial position along the vertical image axis. Each displacement is expressed in terms of pixels. As a result, the lines associated with a same vanishing point undergo displacements of different values with an average of D_{ave}. Then, the vanishing point associated with these displaced lines is determined. The same process is used for determining the second vanishing point. Then we compute the associated focal length. This overall process is repeated for several D_{ave}, as shown in Fig. 14, which gives the variation of the calculated focal length (blue curve) with the average value D_{ave} of the displacements of the ray origins. In this figure, the green curve represents the variation of the error on the calculated focal length. We can see that a displacement around the right position of the ray origins of about -1.5 to 3 pixels does not affect the precision of the calculation of the focal length. This proves that our manual selection of parallel lines is sufficiently precise.

Another way of simulating errors in the process of computing vanishing points is to move one vanishing point away from its precise position (determined by zooming in on the single image 13 as already

explained) in a window of 500×500. After each displacement of this vanishing point, we compute the focal length. The result is plotted in Fig. 15. In this figure, the blue curve represents the calculated focal length as a function of the average value of the displacements of the vanishing points. We can see that an error of 50 pixels is hardly noticeable since it entails an error (green curve) of 0.05 cm on the calculated focal length.

It is true that any camera calibration error (due to the manual selection of sets of parallel lines and to the assumptions made) would affect the 3D reconstruction of potential objects with rectangular boxes. In our case, this problem is not crucial since the position of the rectangular boxes bounding objects can be interactively modified and adjusted to compensate any calibration error.

6 Conclusion and future work

The work presented in this paper contains two contributions: camera calibration and 3D reconstruction. The first one allows the calibration of a camera from a single image (a photograph or any kind of image) containing two vanishing points. The method determines the focal length, then the rotation and translation matrices describing the rigid motion between the world and camera coordinate systems. To compute the translation matrix precisely, we need to know the length of a segment (or edge) in the real scene. To determine the two vanishing points, the user selects two sets of parallel lines. Each set defines a basic vector of the WCS: $\boldsymbol{u}$ and $\boldsymbol{v}$. In the current implementation, the two sets of lines are chosen so that $\boldsymbol{u} \perp \boldsymbol{v}$. Thus, the third basic vector is computed as: $\boldsymbol{w} = \boldsymbol{u} \times \boldsymbol{v}$. Unlike the method described in [14], our approach makes use of two vanishing points (rather than one) and recovers 3D models in a more accurate way. Any kind of image can be used, not necessarily a panoramic image mosaic as in [15].

After the calibration, reconstruction is effected by placing a rectangular box around each potential object; the faces of the box are assigned textures. The holes in the extracted textures corresponding to invisible parts of box faces are filled with other appropriate portions of the image. These textures depend on the single viewpoint. The same textures are used to render the 3D reconstructed model from a novel viewpoint, which may be a source of artifacts as seen on the left side of Fig. 12.

Though the proposed calibration method seems robust, the reconstruction step has to be improved. One solution is to deform a rectangular box (with the graphics user interface) to fit it best with the object to be reconstructed, or to introduce other geometric primitives (such as a: sphere, cylinder, polyhedron, etc.) to bound the objects in the single image. With multiple images, reconstruction could be more precise. Regarding texture mapping, we can do better if multiple images are available, as shown in [7, 9].

We are working on several extensions to enhance our system. For example, if several views are provided, we can calibrate the camera from one image, select a certain number of points on this image, and find the corresponding points on the other images. Next, we determine the fundamental matrices between pairs of images. From these matrices, we compute the 3D transformations (rotation and translation) between pairs of views corresponding to the images. With these transformations, when placing a box around a potential object of one image, this box can be viewed in the other images. This has several advantages. First, box placement becomes more precise. Second, a box's face is assigned several view-dependent textures. Third, we can enhance each texture by adding an out-of-plane displacement component at each texel to get sprites with depths as explained in [29]. Another improvement concerns our graphics user interface. This improvement must help the user to speed up the placement of boxes as well as to extract texture, which requires an outline (delineation) for each box face of parts of the textures corresponding to the object as seen through its box and to make the other parts transparent (using the alpha channel).

References

1. McMillan L, Bishop G (1995) Plenoptic modeling: an image-based rendering system. Proceedings of SIGGRAPH '95, pp 39–46
2. Chen S (1995) Quicktime vr: an image-based approach to virtual environment navigation. Proceedings of SIGGRAPH '95, pp 29–38
3. Gortler SJ, Grzeszczuk R, Szeliski R, Cohen MF (1996) The lumigraph. Proceedings of SIGGRAPH '96, pp 43–54
4. Szeliski R (1996) Video mosaics for virtual environments. IEEE Comput Graph Appl 16:22–30
5. Wood D, Finkelstein A, Hughes J, Salesin D, Thayer C (1997) Multiperspective panoramas for cel animation. Proceedings of SIGGRAPH '97, pp 243–250
6. Rademacher P, Bishop G (1998) Multiple-center-of-projection images. Proceedings of SIGGRAPH '98, pp 199–206

7. Debevec PE, Taylor CJ, Malik J (1996) Modeling and rendering architecture from photographs: A hybrid geometry- and image-based approach. Proceedings of SIGGRAPH '96, pp 11–20
8. Debevec PE, Yu Y, Borshukov GD (1998) Efficient view-dependent image-based rendering with projective texture-mapping. Eurographics Rendering Workshop '98, pp 105–116
9. Poulin P, Ouimet M, Frasson MC (1998) Interactive modeling with photogrammetry. In Drettakis G, Max N (eds) Eurographics Rendering Workshop '98, pp 93–104
10. Faugeras O (1993) Three-dimensional computer vision: a geometric viewpoint. The MIT Press
11. Haralick RM, Shapiro LS (1993) Computer and Robot Vision, volume 2. Addison-Wesley, Reading, Mass
12. Tsai RY (1987) A versatile camera calibration technique for high-accuracy 3D machine vision metrology using off-the-shelf TV cameras and lens. IEEE J Robotics Automat RA-3:323–344
13. Faugeras O, Toscani G (1987) Camera calibration for 3D computer vision. International Workshop on Machine Vision and Machine Intelligence, pp 240–247
14. Horry Y, Anjyo K, Arai K (1997) Tour into the picture: using a spidery mesh interface to make animation from a single image. Proceedings of SIGGRAPH '97, pp 225–232
15. Shum HY, Han M, Szeliski R (1998) Interactive construction of 3D models from panoramic mosaics. In IEEE Conference on Computer Vision and Pattern Recognition (CVPR '98), pp 427–433
16. Parodi P, Piccioli G (1996) 3D shape reconstruction by using vanishing points. Trans Patt Anal Machine Intell 18:211–217
17. Leibowitz D, Criminisi A, Zisserman A (1999) Creating architectural models from images. In: Brunet R, Scopigno R (eds) Eurographics, Vol 18, pp 39–50, Blackwell Publishers
18. Van Den Heuvel FA (1998) 3D reconstruction from a single image using geometric constraints. J Photogrammetry Remote Sensing 53:354–368
19. Haralick RM (1989) Determining camera parameters from the perspective projection of a rectangle. Patt Recogn 22:225–230
20. Caprile B, Torre V (1990) Using vanishing points for camera calibration. Int J Comput Vision 4:127–140
21. Shigang L, Tsuji S, Imai M (1990) Determining of camera rotation from vanishing points of lines on horizontal planes. IEEE 3rd International Conference on Computer Vision (ICCV '90), pp 499–502
22. Chen W, Jiang BC (1991) 3-D camera calibration using vanishing point concept. Comput Vision Patt Recoqn 24:57–67
23. Wang LL, Tsai WH (1991) Camera calibration by vanishing lines for 3-D computer vision. IEEE Trans Patt Anal Machine Intell 13:370–376
24. Szeliski R, Shum HY (1997) Creating full view panoramic image mosaics and environment maps. Proceedings of SIGGRAPH '97, pp 251–258
25. Brillault-O'Mahony B (1991) New method for vanishing point detection. CVGIP: Image Understanding 54:289–300
26. Tai A, Kittler J, Petrou M, Windeatt T (1992) Vanishing point detection. In: Hogg D (ed) British Machine Vision Conference, Springer, Berlin Heidelberg New York, pp 109–118
27. Mark WR, McMillan L, Bishop G (1997) Post-rendering 3D warping. 12th Symposium on Interactive 3D Graphics, pp 7–16
28. Chen E, Williams L (1993) View interpolation for image synthesis. Proceedings of SIGGRAPH '93, pp 279–288
29. Shade J, Gortler S, He L, Szeliski R (1998) Layered depth images. Proceedings of SIGGRAPH '98, pp 241–242

Photographs of the authors and their biographies are given on the next page.

KADI BOUATOUCH He is an electronics and automatic Systems engineer (ENSEM 1974). He was awarded a PhD in 1977 and a higher doctorate on Computer science in the field of Computer graphics in 1989. His is working on global Illumination, lighting Simulation for complex environments, parallel radiosity, augmented reality, visual comfbrt and Computer vision. He applied his research work to infrared and acoustic Simulation. He is currently Professor at the university of Rennes l (France) and researcher at IRISA (Institut de Recherche en Informatique et Systemes aleatoires). He is member of Eurographics, ACM and IEEE.

ERWAN GUILLOU He took a postgraduate degree in Computer science in 1996 from the university of Rennes l. He is currently preparing at IRISA a PhD thesis in the field of Computer graphics. His interest is Computer vision and augmented reality.

DANIEL MENEVEAUX He received a PhD in Computer science in 1998 from the university of Rennes l. His interest is: sequential and parallel radiosity, partioning of large scenes, visibility, image based rendering, light field. He is currently lecturer at the university of Poitiers.

ERIC MAISEI He received a PhD in Computer science in 1992 from the university of Rennes l. His is working on global illumination in complex environments, radiosity, augmented reality and Computer vision. He is currently lecturer at the university of Rennes l and researcher at IRISA.